Cell Identitas

```
NAMA = 'Muhammad Pearl Ocshada'
NIM = '244107020064'
print(NAMA, NIM)
```

```
Muhammad Pearl Ocshada 244107020064
```

Praktikum 7 - Segmentasi Citra dengan Thresholding

D1 - Tugas Individu

A. Load Data

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

B. Import Library yang digunakan

```
import cv2 as cv
from google.colab.patches import cv2_imshow
import numpy as np
import math
import pandas as pd
import matplotlib.pyplot as plt
from collections import deque
from scipy import ndimage as ndi
from skimage import data, filters, color, segmentation, feature, morphology
from skimage.filters import gaussian
from skimage.segmentation import flood
```

C. Membuat Global Thresold

- BINARY
- BINARY_INV
- TRUNC
- TOZERO
- TOZERO_INV

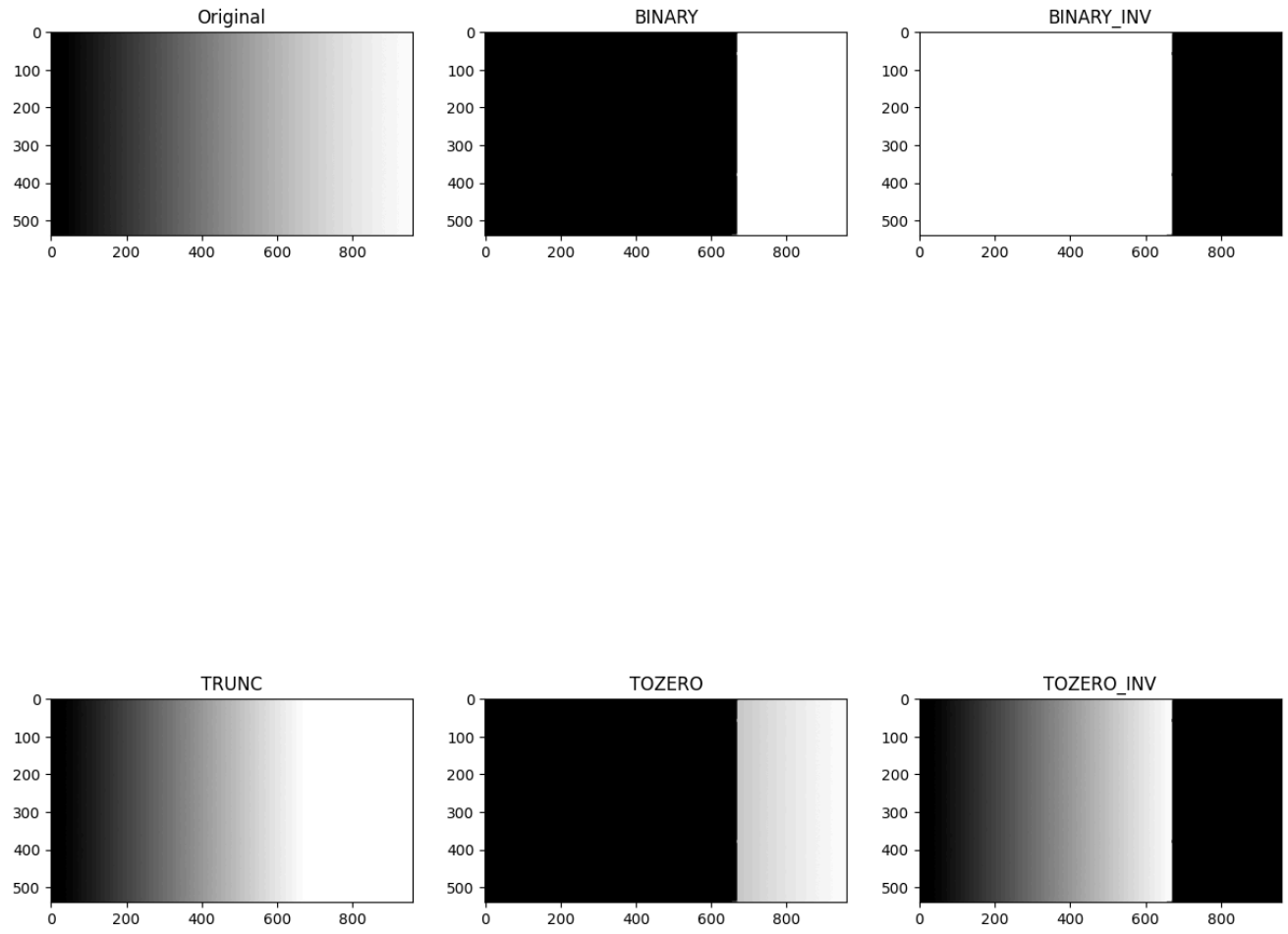
```
img = cv.imread('/content/drive/MyDrive/PCVK/P7/gradient.jpg', 0) #0 - membaca sebagai greyscale (1 channel)
t = 200

mask = img > t

binary = np.where(mask, 255, 0).astype('uint8')
binary_inv = np.where(mask, 0, 255).astype('uint8')
trunc = np.where(mask, t, img).astype('uint8')
tozero = np.where(mask, img, 0).astype('uint8')
tozero_inv = np.where(mask, 0, img).astype('uint8')

images = [img, binary, binary_inv, trunc, tozero, tozero_inv]
titles = ['Original', 'BINARY', 'BINARY_INV', 'TRUNC', 'TOZERO', 'TOZERO_INV']

plt.figure(figsize=(15, 15))
for i in range(6):
    plt.subplot(2, 3, i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
plt.show()
```



D. Membuat Adaptive Threshold

- Global Threshold (T=127)
- Adaptive Mean - Manual
- Adaptive Gaussian - Manual

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

# Membaca gambar dalam format grayscale
img = cv.imread('/content/drive/MyDrive/PCVK/P7/sudoku.png', 0)

# Inisialisasi parameter
t = 127
window = 11
c = 2
sigma = 2

# 1. Global Threshold
global_thresh = np.where(img > t, 255, 0).astype('uint8')

# Pad image untuk menangani area pinggir (border)
pad_size = window // 2
img_padded = np.pad(img, pad_size, mode='edge')

h, w = img.shape
adaptive_mean = np.zeros_like(img)
adaptive_gaussian = np.zeros_like(img)

# Membuat kernel Gaussian manual
x, y = np.mgrid[-pad_size:pad_size+1, -pad_size:pad_size+1]
```

```
g_kernel = np.exp(-(x**2 + y**2) / (2 * sigma**2))
g_kernel = g_kernel / g_kernel.sum()

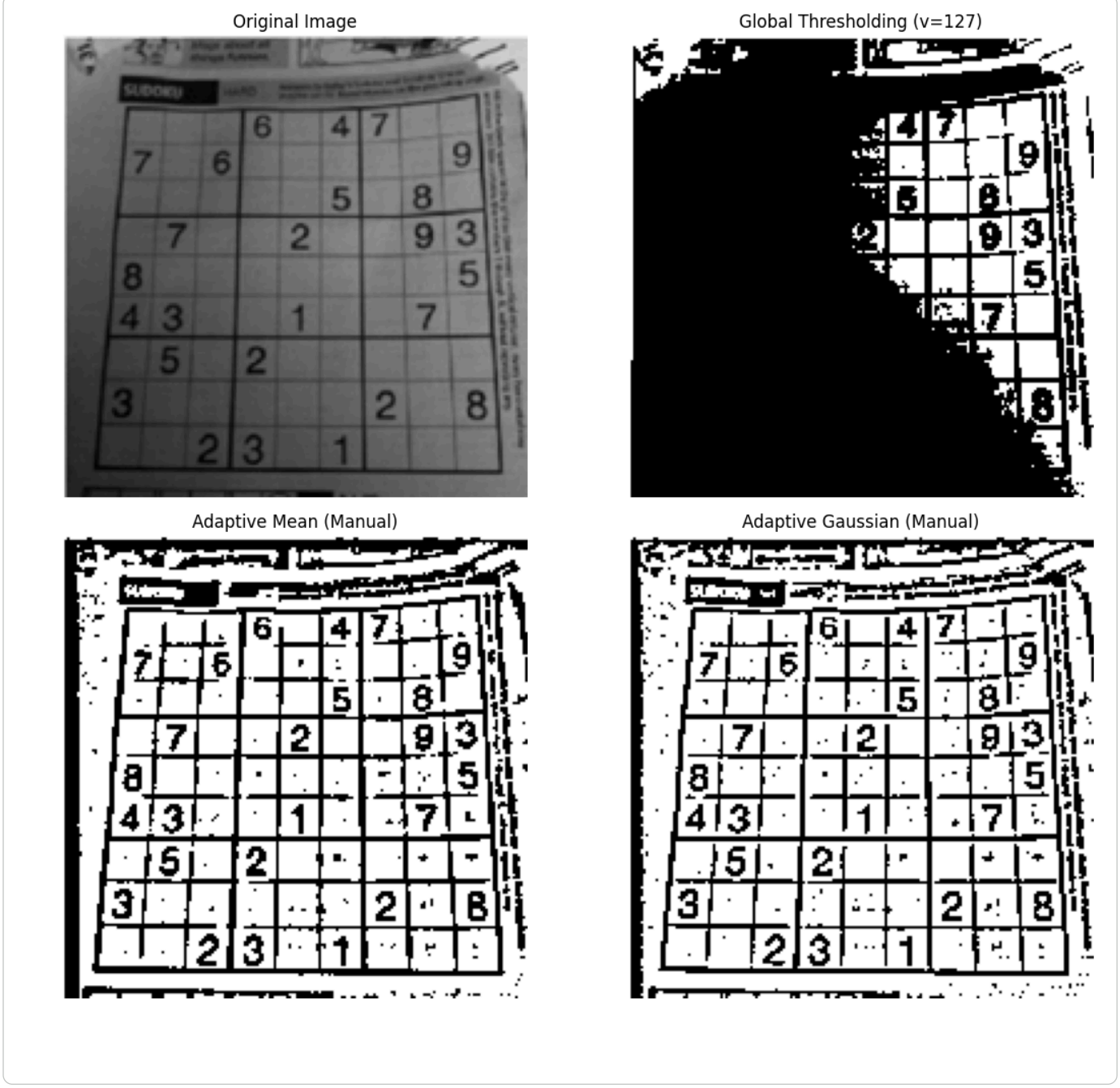
# Perhitungan manual menggunakan sliding window
for i in range(h):
    for j in range(w):
        # Ambil wilayah lokal (neighborhood)
        local_region = img_padded[i:i+window, j:j+window]

        # Rumus Manual Adaptive Mean: Rata-rata lokal - C
        mean_val = np.mean(local_region)
        thresh_mean = mean_val - c
        adaptive_mean[i, j] = 255 if img[i, j] > thresh_mean else 0

        # Rumus Manual Adaptive Gaussian: Weighted sum dengan kernel Gaussian - C
        gaussian_val = np.sum(local_region * g_kernel)
        thresh_gaussian = gaussian_val - c
        adaptive_gaussian[i, j] = 255 if img[i, j] > thresh_gaussian else 0

# Menampilkan hasil visualisasi
titles = ['Original Image', 'Global Thresholding (v=127)', 'Adaptive Mean (Manual)', 'Adaptive Gaussian (Manual)']
images = [img, global_thresh, adaptive_mean, adaptive_gaussian]

plt.figure(figsize=(12, 10))
for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
plt.tight_layout()
plt.show()
```



E. Buat Otsu Threshold tanpa library dan tampilkan nilai threshold menggunakan otsu's

```
img = cv.imread('/content/drive/MyDrive/PCVK/P7/lena.jpg',0)
blur = cv.GaussianBlur(img,(5,5),0)
t=127


def otsu(gray):
    pixel_number = gray.shape[0] * gray.shape[1]
    mean_weight = 1.0/pixel_number
    his, bins = np.histogram(gray, np.arange(0,257))
    final_thresh = -1
    final_value = -1
    intensity_arr = np.arange(256)
    for t in bins[1:-1]:
        pcb = np.sum(his[:t])
        pcf = np.sum(his[t:])
        if pcb == 0 or pcf == 0:
            continue
        Wb = pcb * mean_weight
        Wf = pcf * mean_weight
```

```
            mub = np.sum(intensity_arr[:t]*his[:t]) / float(pcb)
            muf = np.sum(intensity_arr[t:]*his[t:]) / float(pcf)
            value = Wb * Wf * (mub - muf) ** 2

            if value > final_value:
                final_thresh = t
                final_value = value
        final_img = gray.copy()
        print("Otsu Threshold:", final_thresh)
        final_img[gray > final_thresh] = 255
        final_img[gray <= final_thresh] = 0
        return final_img, final_thresh

    otsu_biner, otsu_thresh = otsu(blur)
    x = ("Otsu's Thresholding (v = ")+str(otsu_thresh)+")"
    ret,th1 = cv.threshold(blur,thresh,255,cv.THRESH_BINARY)
    ret, otsu_library = cv.threshold(blur,0,255,cv.THRESH_BINARY + cv.THRESH_OTSU)

    titles = ['Citra Asli', 'Global Thresholding (Thresh = ' + str(thresh) + ')', ' Otsu Manual (v = ' + str(otsu_thresh) + ')', 'O
    citra3 = [blur, th1, otsu_biner, otsu_library]

    plt.figure(figsize = (12, 10))
    for i in range(len(citra3)):
        plt.subplot(2, 2, i+1)
        plt.imshow(citra3[i], 'gray')
        plt.title(titles[i])
        plt.xticks([]), plt.yticks([])
    plt.tight_layout()
    plt.show()
```

```
Otsu Threshold: 114
```

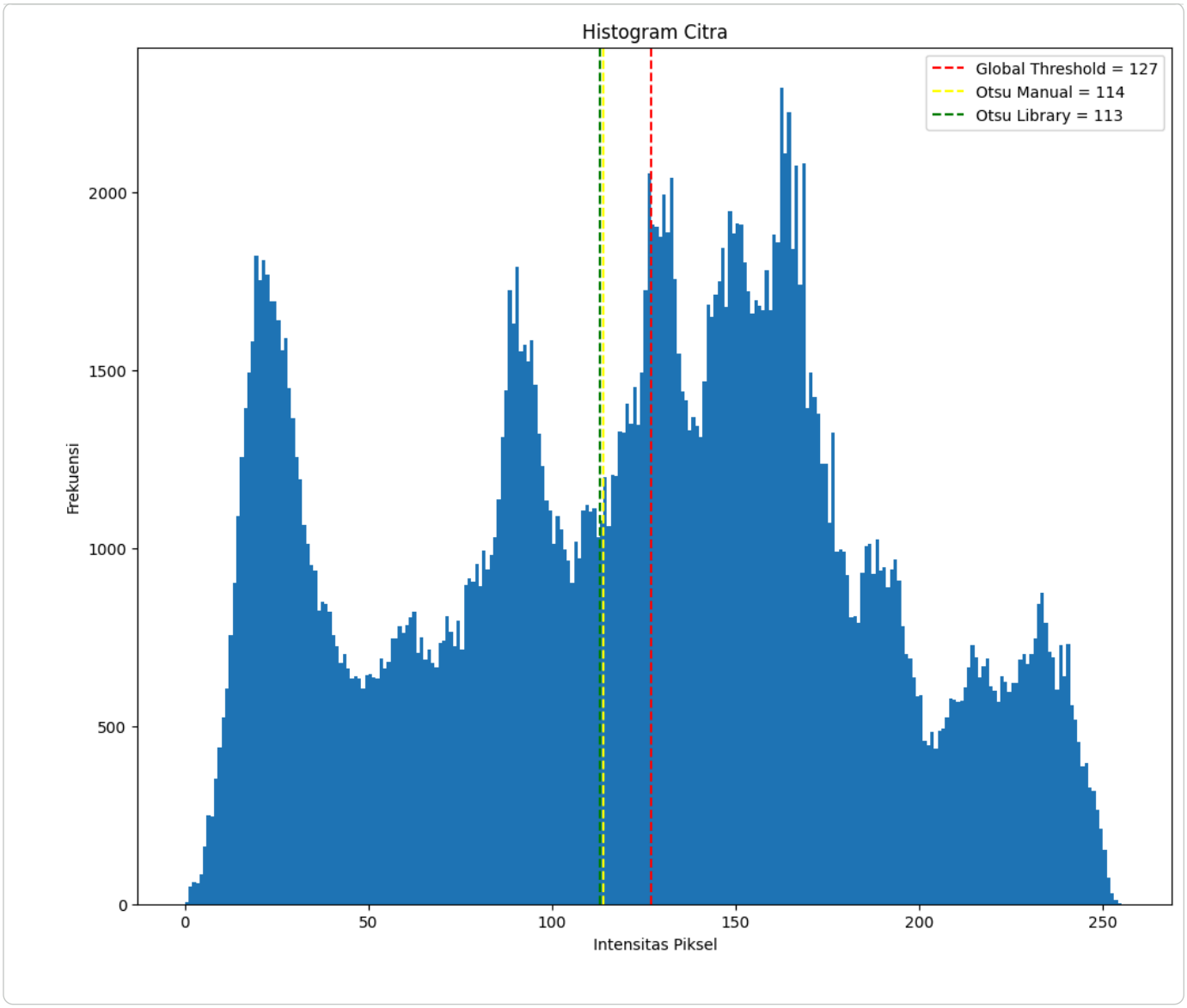



```
plt.figure(figsize=(12, 10))

plt.hist(blur.ravel(), bins=256, range=[0, 256])

plt.axvline(127, color='red', linestyle='--', label='Global Threshold = 127')
plt.axvline(114, color='yellow', linestyle='--', label='Otsu Manual = 114')
plt.axvline(113, color='green', linestyle='--', label='Otsu Library = 113')

plt.title('Histogram Citra')
plt.xlabel('Intensitas Piksel')
plt.ylabel('Frekuensi')
plt.legend()
plt.show()
```



Jawaban : supaya foto Lena masih terlihat, T harus berada **di dalam rentang histogram** (di antara min dan max, kira-kira 86–107). Nilai yang paling baik ada di sekitar puncak pemisah dua kelompok, yaitu ±90–95 (Otsu = 93). T terlalu dekat min ⟶ hampir putih semua; T terlalu dekat max ⟶ hampir hitam semua.

F. Coba dengan IMG KTP agar terlihat beda antara otsu's -> global threshold

```
# Membaca citra KTM dalam format grayscale
img = cv.imread('/content/drive/MyDrive/PCVK/Images/244107020064/KTM1b.jpg', 0)

# Lakukan Gaussian Blur
blur = cv.GaussianBlur(img, (5, 5), 0)

t = 127

# Global Thresholding
global_thres = np.where(blur > t, 255, 0).astype('uint8')

# Otsu Thresholding (Menggunakan fungsi manual otsu yang sudah didefinisikan sebelumnya)
otsu_manual, otsu_thresh = otsu(blur)
#Otsu Library
ret, otsu_library = cv.threshold(blur, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)

# Menyiapkan data visualisasi khusus untuk KTM
ktm_images = [img, global_thres, otsu_manual, otsu_library]
ktm_titles = ['KTM Asli', 'Global Thresholding (v = 127)', f'Otsu Manual (v = {otsu_thresh})', f'Otsu Library (v = {int(ret)})']

plt.figure(figsize=(12, 10))
```

```
for i in range(len(ktm_images)):
    plt.subplot(2, 2, i+1)
    plt.imshow(ktm_images[i], 'gray')
    plt.title(ktm_titles[i])
    plt.xticks([]), plt.yticks([])
plt.tight_layout()
plt.show()
```

```
Otsu Threshold: 176
```



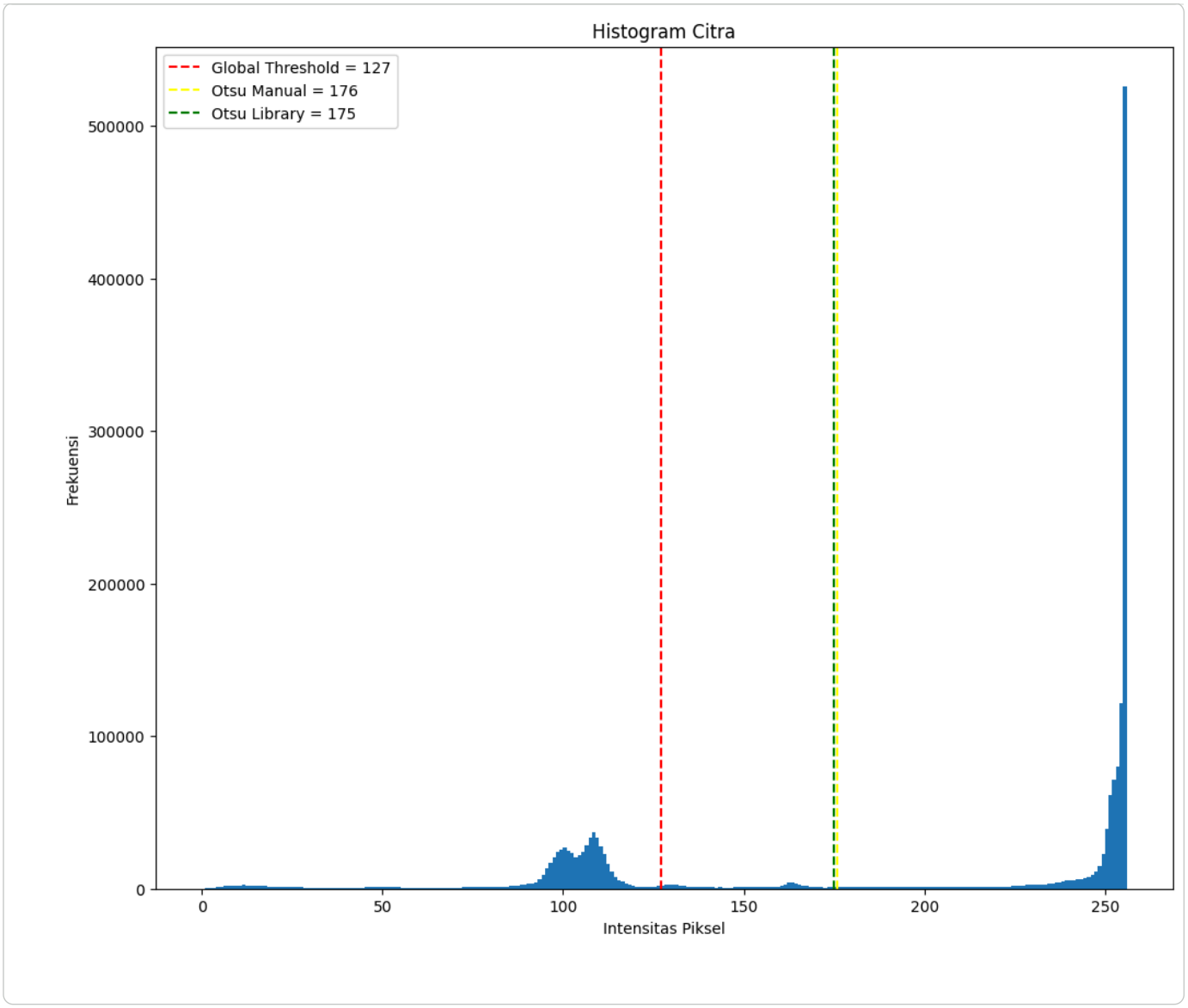
Membuat Histogram

```
plt.figure(figsize=(12, 10))

plt.hist(blur.ravel(), bins=256, range=[0, 256])

plt.axvline(127, color='red', linestyle='--', label='Global Threshold = 127')
plt.axvline(176, color='yellow', linestyle='--', label='Otsu Manual = 176')
plt.axvline(175, color='green', linestyle='--', label='Otsu Library = 175')

plt.title('Histogram Citra')
plt.xlabel('Intensitas Piksel')
plt.ylabel('Frekuensi')
plt.legend()
plt.show()
```



G. Segmentasi berbasis Region

```
img = data.coins()


seed_list = {
    "Dalam Koin": (50, 45),
    "Background": (20, 20),
    "Tepi Koin": (40, 62)
}


NEIGHBOR_8 = [
    (-1, -1), (-1, 0), (-1, 1),
    (0, -1),           (0, 1),
    (1, -1),  (1, 0),  (1, 1)
]

def grow_region(image, seed, threshold):

    height, width = image.shape

    # False = belum menjadi bagian region
    region = np.zeros((height, width), dtype=bool)

    # Queue untuk BFS
    queue = deque()
    queue.append(seed)
```

```
        # Seed langsung dimasukkan ke region
        region[seed] = True

        # Statistik region
        total_intensity = float(image[seed])
        pixel_count = 1

        while queue:

            current_r, current_c = queue.popleft()

            # Hitung rata-rata region saat ini
            region_mean = total_intensity / pixel_count

            # Periksa 8 tetangga
            for dr, dc in NEIGHBOR_8:

                new_r = current_r + dr
                new_c = current_c + dc

                # Pastikan koordinat masih berada di dalam gambar
                if not (0 <= new_r < height and 0 <= new_c < width):
                    continue

                # Jika pixel sudah masuk region, lewati
                if region[new_r, new_c]:
                    continue

                # Nilai pixel kandidat
                pixel_value = float(image[new_r, new_c])

                # Cek kemiripan dengan region
                difference = abs(pixel_value - region_mean)

                if difference < threshold:

                    # Masukkan pixel ke region
                    region[new_r, new_c] = True
                    queue.append((new_r, new_c))

                    # Update statistik region
                    total_intensity += pixel_value
                    pixel_count += 1

        return region, pixel_count

    def create_overlay(image, region):

        # Grayscale → RGB
        overlay = cv.cvtColor(image, cv.COLOR_GRAY2RGB)

        # Region hasil diberi warna merah
        overlay[region] = [255, 0, 0]

        return overlay

    threshold_values = [5, 20, 60]

    fig, axes = plt.subplots(
        len(seed_list),
        len(threshold_values) + 1,
        figsize=(16, 10)
    )


    for row, (seed_name, seed_point) in enumerate(seed_list.items()):

        ax = axes[row, 0]

        ax.imshow(img, cmap='gray')

        ax.scatter(
            seed_point[1],
            seed_point[0],
            color='red',
            s=40
        )
```

```
        ax.set_title(
            f"Citra Asli\n{seed_name}"
        )

        ax.axis('off')

        for col, threshold in enumerate(threshold_values, start=1):

            region, pixel_count = grow_region(
                img,
                seed_point,
                threshold
            )

            result = create_overlay(
                img,
                region
            )

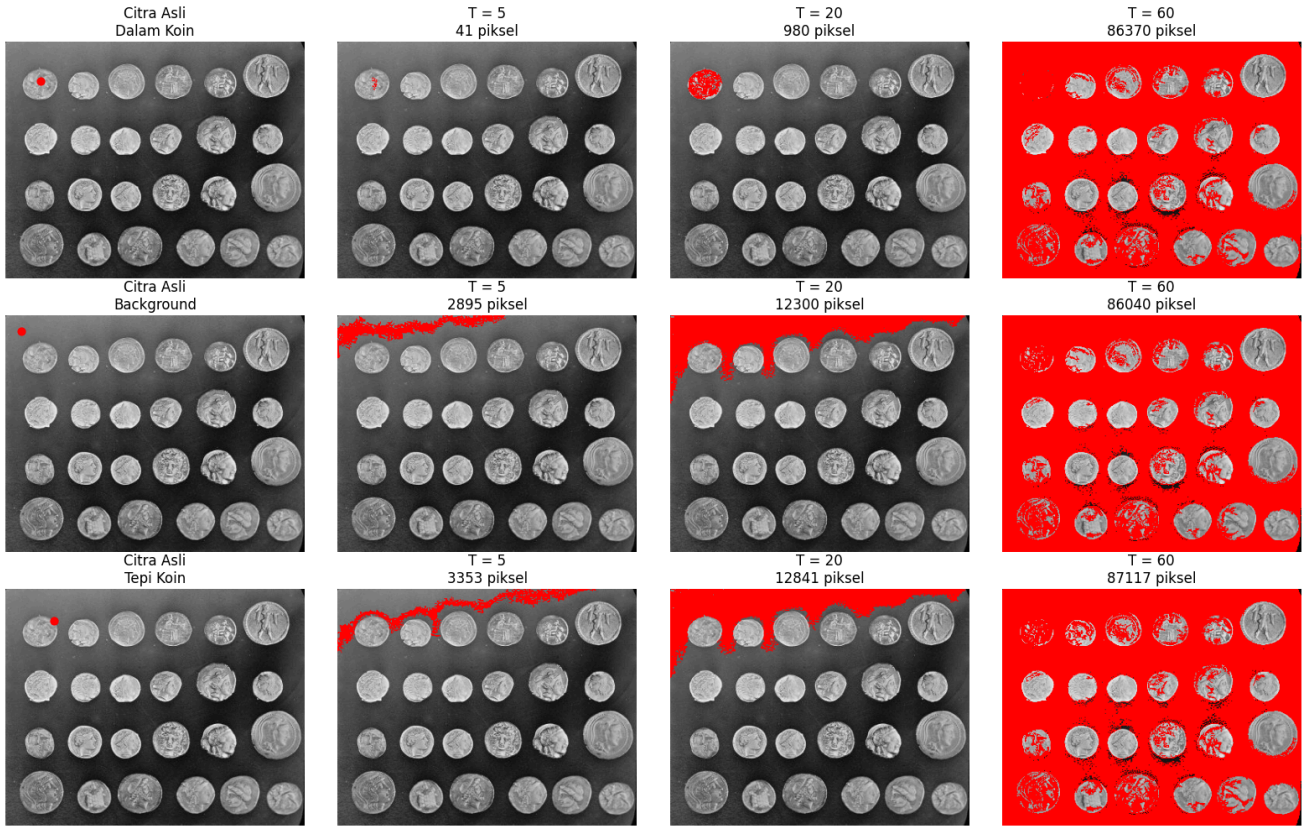
            ax = axes[row, col]

            ax.imshow(result)

            ax.set_title(
                f"T = {threshold}\n"
                f"{pixel_count} piksel"
            )

            ax.axis('off')


plt.tight_layout()
plt.show()
```



H. Segmentasi Berbasis Region dengan flood

```
# Citra coins
gray = data.coins()

# Seed
seed = (36, 43)

# Nilai toleransi
tolerances = [5, 20, 60]

# Menampilkan informasi seed
print(f"Seed (baris, kolom): {seed} | intensitas: {gray[seed]:.6f}")

plt.figure(figsize=(16, 4))

# Citra asli + seed
plt.subplot(1, 4, 1)
plt.imshow(gray, cmap='gray')
plt.scatter(seed[1], seed[0], c='red', s=40)
plt.title('Citra asli + seed')
plt.xticks([])
plt.yticks([])


# Flood untuk setiap tolerance
for i, T in enumerate(tolerances):

    region = flood(
        gray,
        seed_point=seed,
        tolerance=T,
        connectivity=1
    )

    # Menghitung jumlah piksel region
    jumlah_piksel = np.sum(region)

    # Membuat citra RGB
    hasil = cv.cvtColor(gray, cv.COLOR_GRAY2RGB)

    # Memberikan warna merah pada region
    hasil[region] = [255, 0, 0]

    plt.subplot(1, 4, i + 2)

    plt.imshow(hasil)

    plt.title(
        f'Region Growing, T = {T}\n'
        f'({jumlah_piksel} piksel)'
    )

    plt.xticks([])
    plt.yticks([])
```